



UNIVERSITY OF RWANDA

College Of Science and Technology

Computer Engineering

Year *4*

Gako Campus

Reg nº: **223026685**

223026674

2230267475

Names: Jerome HAKIZIMANA

Lewis NGENDAHIMANA

Clement NZABAMWITA

Contents

1. Introduction 2
2. Mini Project 2.1: Simulating RM, EDF and Priority Inheritance 2
2.1 Objective 2
2.2 Method 3
2.3 Results 3
Utilization tests and RTA 3
RM and EDF schedules over the hyperperiod (24 ticks) 4
Priority inversion and PIP 4
Lab tasks 5
2.4 Why simulated PIP blocking matches the bound 6
3. Mini Project 2.2: RM versus EDF Schedulability Comparison Tool 6
3.1 Objective 6
3.2 Design 6
3.3 Task sets and results 7
3.4 Written report 7
3.5 Reflection: when the bound fails but RTA succeeds 11
4. Mini Project 2.3: Partitioned Multiprocessor Scheduling Simulator 11
4.1 Objective 11
4.2 Method 11
4.3 Results 11
4.4 Analysis 13
5. Conclusions 14
Limitations 14
Delivered files 14

GIT LINK: https://github.com/sticker2024/RTS-LAB

Real-Time Systems

Unit 2 Lab Report: Scheduling, Schedulability Analysis and Partitioning

Course	Real-Time Systems, Mini Projects
Unit	Unit 2 (rate-monotonic and EDF scheduling, priority inversion, multiprocessor partitioning)
Mini projects	2.1 Simulating RM, EDF and Priority Inheritance in Python; 2.2 RM versus EDF Schedulability Comparison Tool; 2.3 Partitioned Multiprocessor Scheduling Simulator
Date	27 October 2026
Name / Student ID	**1. Jerome HAKIZIMANA** 223026685 **2. Lewis NGENDAHIMANA** 223026674 **3. Clement NZABAMWITA** 2230267475

1. Introduction

This report documents the three Unit 2 mini projects. All three are pure-Python simulations with matplotlib charts; no RTOS or hardware is involved. Time advances in whole ticks, a job of task τ_i is released whenever the tick is a multiple of T_i, and every result in this report was produced by the scripts delivered with it, so each number can be reproduced by re-running them.

- **Project 2.1** builds the simulator module `rts_sim.py`: utilization tests, Rate Monotonic (RM), Earliest Deadline First (EDF) and Least Laxity First (LLF) scheduling, Response Time Analysis (RTA), and the priority-inversion scenario with and without the Priority Inheritance Protocol (PIP).
- **Project 2.2** wraps that module in a general schedulability tool (`schedulability_tool.py`) that accepts a task list or CSV file and compares RM and EDF analytically and by simulation.
- **Project 2.3** adds a partitioning simulator (`partition.py`) that assigns tasks to cores with First-Fit (FF), First-Fit Decreasing (FFD), Best-Fit Decreasing (BFD) and Worst-Fit (WF), using the Liu & Layland bound as the per-core admission test.

2. Mini Project 2.1: Simulating RM, EDF and Priority Inheritance

2.1 Objective

To implement a discrete-event simulator for periodic tasks under RM and EDF, to apply the utilization tests before simulating, to visualize the schedules as Gantt charts, and to reproduce the priority-inversion scenario together with its PIP remedy.

2.2 Method

- A `Task` record holds the name, C, T and D (D defaults to T), with U = C/T. `rm_bound(n)` returns n(2^(1/n) − 1).
- A single tick-by-tick engine releases jobs, marks a job as a deadline miss if it is unfinished at its absolute deadline, and asks a selection rule which ready job runs. RM picks the shortest period, EDF the smallest absolute deadline, and LLF the smallest laxity (deadline − now − remaining work). Deadline Monotonic (DM) reuses the RM engine with tasks ordered by relative deadline.
- RTA iterates $R_i = C_i + \Sigma\lceil R_i/T_j \rceil C_j$ over higher-priority tasks from $R_i = C_i$ until it converges or exceeds D_i.
- Priority-inversion model (Section 2.6.1): L (lowest priority) owns the shared resource from t = 0 and needs 6 ticks inside the critical section; H (highest) arrives at t = 1 and needs the resource for 2 ticks; M (medium) arrives at t = 2 and needs 4 ticks without the resource. With PIP, when H blocks, L inherits H's priority until it releases the resource.

2.3 Results

Utilization tests and RTA

Table 1: Utilization tests for the two worked examples

Task set	n	U	n(2^(1/n) – 1)	RM sufficient test	EDF (U ≤ 1)
3 tasks, Section 2.2.2: (1,4), (2,6), (2,12)	3	0.750	0.780	PASS	PASS
4 tasks, Section 2.3.2: adds (1,8)	4	0.875	0.757	Inconclusive	PASS

Both worked examples reproduce the figures of Sections 2.2.2 and 2.3.2 exactly. For the four-task set the bound is inconclusive, so RTA gives the definitive answer (Table 2). In priority order the tasks are τ_1, τ_2, τ_4, τ_3, and every response time is within its deadline, including $R_4 = 4 \le D_4 = 8$. The tight task is τ_3, which responds at 11 against a deadline of 12 after five iterations.

Table 2: Response Time Analysis for the four-task set (RM priority order)

Task	C	T	D	R_i	Iterations	$R_i \le D_i$
τ_1	1	4	4	1	1	yes
τ_2	2	6	6	3	2	yes
τ_4	1	8	8	4	2	yes
τ_3	2	12	12	11	5	yes

RM and EDF schedules over the hyperperiod (24 ticks)

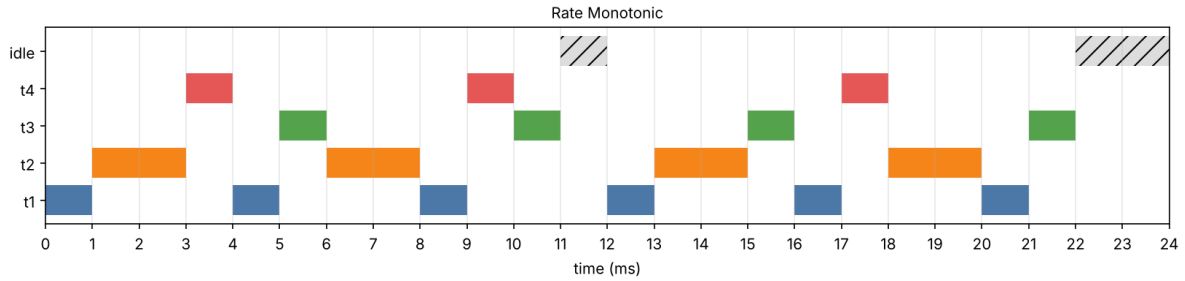


Figure 1: *Rate Monotonic schedule of the four-task set over its 24-tick hyperperiod.*

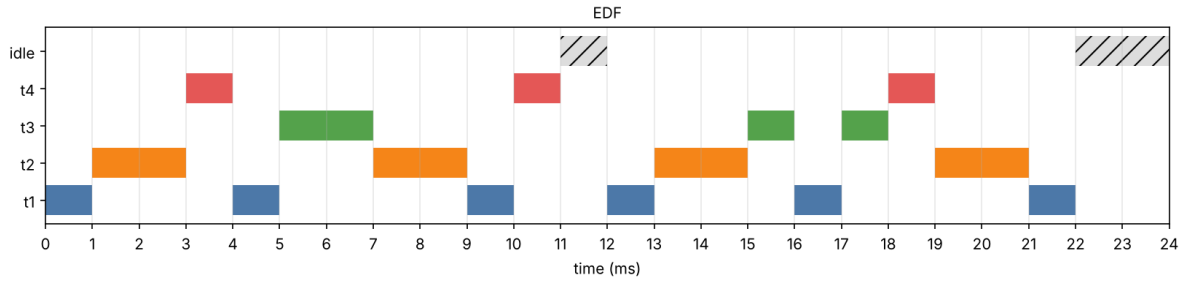


Figure 2: *Earliest Deadline First schedule of the same set.*

Neither policy misses a deadline. Total idle time is 3 of 24 ticks (1 − 0.875 = 12.5%) under both. The charts agree on τ_1 and diverge among the others: under RM the first job of τ_3 is split around higher-priority arrivals and completes at t = 11, which is exactly the RTA value, whereas under EDF it runs at t = 5 to 7 and completes at t = 7 because its deadline is nearer than the competing jobs'. Over one hyperperiod RM makes 15 context switches and EDF 14.

Priority inversion and PIP

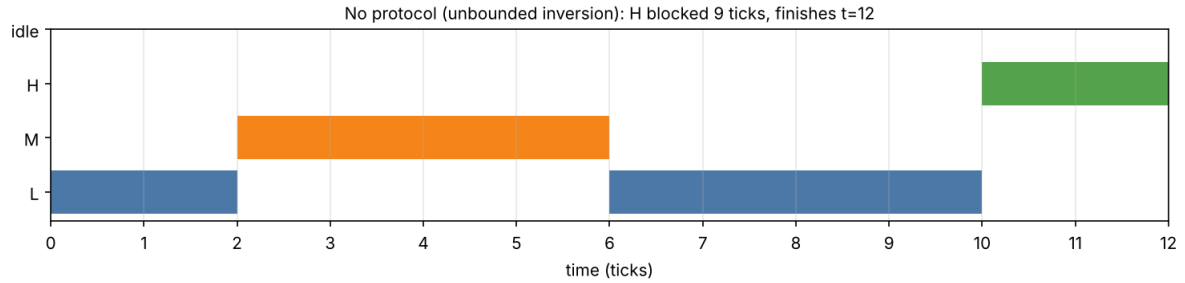


Figure 3: *No protocol: M preempts L inside the critical section, so H waits for L's work and all of M.*

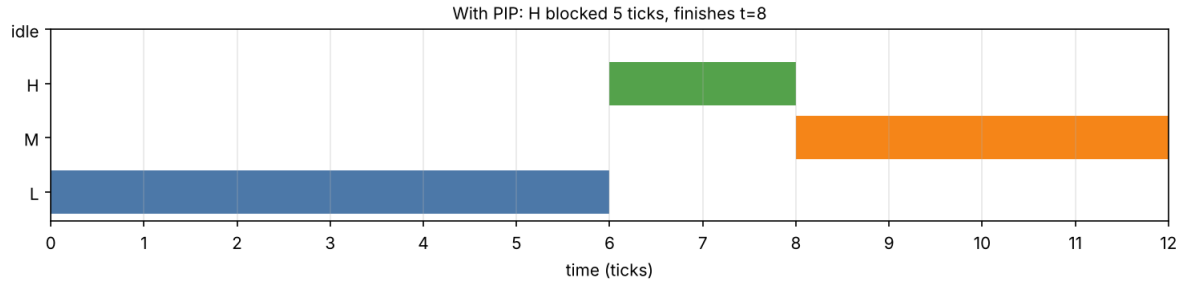


Figure 4: *With PIP: L inherits H's priority, finishes its section by t = 6, H runs at t = 6 to 8, then M.*

Without a protocol L runs at t = 0 to 2, M preempts it at t = 2 and runs to t = 6, and L only finishes its section at t = 10, so H waits 9 ticks and completes at t = 12. With PIP, L inherits H's priority at t = 1, M cannot preempt, L releases the resource at t = 6, and H runs at t = 6 to 8 after being blocked for 5 ticks.

Lab tasks

- **Deadline Monotonic.** For an implicit-deadline set the DM key (D, T) equals the RM key (T, D), so the schedule cannot change. On the constrained set a (C = 1, T = 5, D = 2), b (2, 8, 8), c (1, 10, 3), RM orders the tasks a, b, c and task c misses its deadline of 3; DM orders them a, c, b and no deadline is missed.
- **Bound sweep.** The RM bound falls from 1.000 at n = 1 through 0.828 (n = 2), 0.780 (n = 3) and 0.757 (n = 4) to 0.718 at n = 10, converging toward ln 2 ≈ 0.693 while the EDF bound stays at 1.0 (Figure 5).
- **Least Laxity First.** Over the same hyperperiod LLF makes 17 context switches against 15 for RM and 14 for EDF, with no deadline miss: it schedules the set correctly but changes jobs more often. A switch here means any change of executing job between consecutive busy ticks.
- **Additional medium-priority tasks.** A second medium task M2 (arrives t = 4, needs 4 ticks) and a third M3 (t = 6, 4 ticks) were added. Results are in Table 3 and Figure 6 and 7.

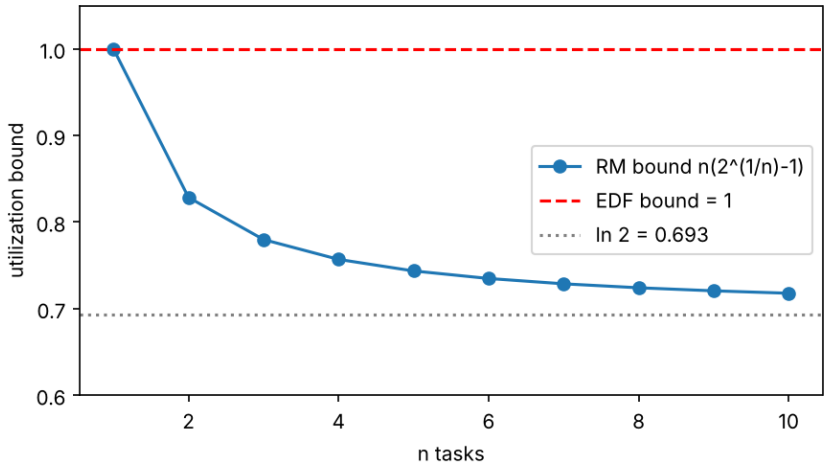


Figure 5: *Liu & Layland bound for n = 1 to 10 against the EDF bound of 1.0 and ln 2.*

Table 3: Blocking time of H as medium-priority tasks are added (ticks)

Medium tasks	No protocol: H blocked	No protocol: H completes	PIP: H blocked	PIP: H completes
M1	9	t = 12	5	t = 8
M1, M2	13	t = 16	5	t = 8
M1, M2, M3	17	t = 20	5	t = 8

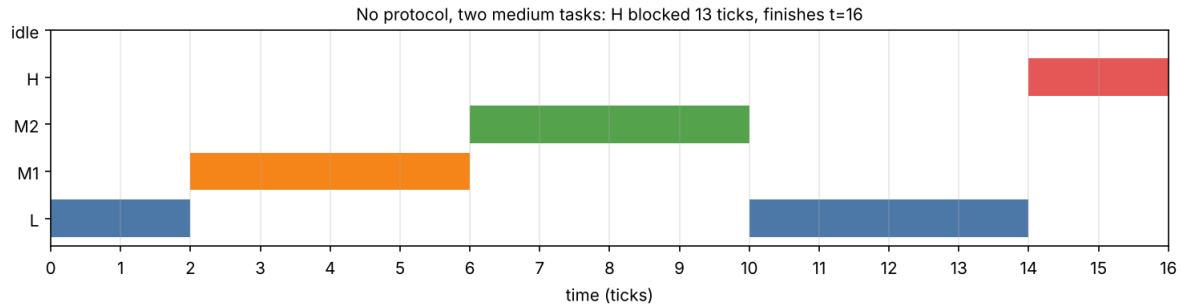


Figure 6: *No protocol with two medium tasks: M1 and M2 both run before L can finish its section.*

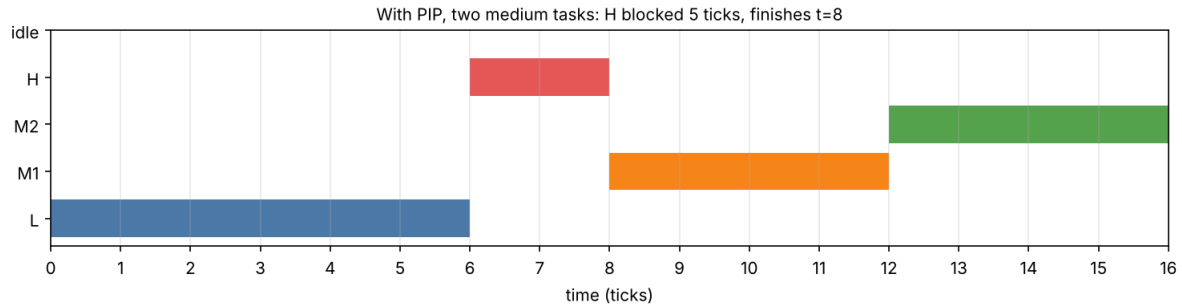


Figure 7: *PIP with two medium tasks: H's blocking is unchanged; M1 and M2 are deferred until after H.*

2.4 Why simulated PIP blocking matches the bound

Under the Priority Inheritance Protocol, H can be blocked only while L holds the resource, and as soon as H starts waiting, L inherits H's priority. From that moment M, whose priority is below H's, can no longer preempt L, so the unrelated medium-priority work is deferred until H has finished. H's blocking time therefore equals the time L still needs to complete its critical section. L's section is 6 ticks and L had already executed one tick when H arrived at t = 1, so H waited ticks 1 to 5, a measured blocking time of 5 ticks, and then ran at t = 6 to 8. This is consistent with Section 2.7.1: H is blocked for at most one critical section of each lower-priority task that shares its resource, and here there is exactly one such task, so the bound is 6 ticks. The measurement sits one tick under the bound only because H arrived after L had started; the bound is reached when H arrives at the instant L enters the section. The decisive evidence is the comparison: without PIP the blocking was 9 ticks with one medium task and grew to 13 and 17 with two and three, because each adds its full execution time, whereas under PIP it stayed at 5 in every case.

3. Mini Project 2.2: RM versus EDF Schedulability Comparison Tool

3.1 Objective

To build a general tool that takes any periodic task set, reports its schedulability under RM and EDF using the tests of Sections 2.2, 2.3 and 2.5, and draws a timeline comparison of the two schedules.

3.2 Design

- **Input:** an in-script list of `Task` objects, or a CSV file with columns name, C, T and an optional D (`python schedulability_tool.py tasks.csv --out chart.png`). Invalid input (missing

file, non-positive values, D > T) produces a clear error message and a non-zero exit code instead of a crash.

- **Decision layer:** compute u_i and U; if U > 1 report the set as infeasible; for implicit deadlines apply the Liu & Layland bound and, only when it fails, run RTA per task; for constrained deadlines use DM ordering and RTA directly; judge EDF by U ≤ 1 (with the density sum ΣC/D reported for constrained sets).
- **Confirmation and chart:** RM (or DM) and EDF are simulated over the hyperperiod, capped at 500 ticks, and drawn as a two-panel Gantt chart with idle time marked.

3.3 Task sets and results

Table 4: Task sets used (C, T and, for constrained sets, D)

Set	Tasks
A	Section 2.2.2: (1, 4), (2, 6), (2, 12)
B	Section 2.3.2: (1, 4), (2, 6), (2, 12), (1, 8)
C	Bound fails but RTA succeeds: (2, 5), (4, 10), (1, 20)
D	Overload, U > 1: (3, 4), (2, 5)
E	Constrained deadlines: a (1, 5, D = 2), b (2, 8, D = 8), c (1, 10, D = 3)
F	Eight tasks: (1, 10), (1, 12), (3, 20), (1, 15), (3, 30), (4, 40), (3, 25), (1, 50)

Table 5: Analytical verdicts and simulation check

Set	n	Deadlines	U	Bound	Bound test	RTA (R against D)	EDF	Misses RM/DM, EDF
A	3	implicit	0.750	0.780	PASS	not needed	PASS	0, 0
B	4	implicit	0.875	0.757	Inconclusive	R = 1, 3, 4, 11 vs D = 4, 6, 8, 12: schedulable	PASS	0, 0
C	3	implicit	0.850	0.780	Inconclusive	R = 2, 8, 9 vs D = 5, 10, 20: schedulable	PASS	0, 0
D	2	implicit	1.150	0.828	Infeasible (U > 1)	not run	FAIL	4, 3
E	3	constrained	0.550	n/a	Skipped (D < T)	DM: R = 1, 2, 4 vs D = 2, 3, 8: schedulable	U ≤ 1 passes; density 1.083	0, 0
F	8	implicit	0.740	0.724	Inconclusive	all eight meet D; largest R = 20 (D = 50)	PASS	0, 0 (500 of 600 ticks)

3.4 Written report

The tool follows the decision procedure of Sections 2.2.1, 2.3.1 and 2.5.2. For each set it computes every u_i and the total U, and declares a set with U > 1 infeasible at once. For implicit deadlines it applies the Liu & Layland bound n(2^(1/n) − 1) and, only if that test fails, runs RTA per task in priority order. For constrained deadlines it switches to Deadline Monotonic ordering, skips the bound (which assumes D = T) and goes straight to RTA. EDF is judged by the exact condition U ≤ 1, and the density sum $\Sigma C_i/D_i$ is reported as an extra sufficient test when deadlines are constrained. Every verdict is cross-checked by simulating both policies over one hyperperiod, capped at 500 ticks.

Set A (Section 2.2.2) has U = 0.750 against a bound of 0.780, so RM schedulability is guaranteed without RTA, and EDF passes too. **Set B** (Section 2.3.2) has U = 0.875, above the bound of 0.757, so

the sufficient test is inconclusive and the tool falls back to RTA. The responses are 1, 3, 4 and 11 for τ_1, τ_2, τ_4 and τ_3 against deadlines 4, 6, 8 and 12, so RM does schedule the set, although τ_3 has only one tick of slack. **Set C** has U = 0.850 against 0.780; the bound fails but RTA gives 2, 8 and 9 against deadlines 5, 10 and 20, so the set is RM-schedulable, as discussed in the reflection below.

Set D has U = 1.150. The tool reports it as infeasible for any uniprocessor scheduler and does not run RTA. Simulation confirms this: over 20 ticks RM misses four deadlines and EDF three, so EDF's optimality gives no protection against overload. **Set E** has constrained deadlines. Its U is only 0.550, but DM ordering (a, c, b) is required: under RM ordering task c would miss its deadline of 3, whereas RTA under DM gives 1, 2 and 4 against 2, 3 and 8. For EDF the condition $U \leq 1$ is only necessary here, and the density sum of 1.083 means the sufficient test is inconclusive, yet the simulation shows no miss. **Set F** has eight tasks and U = 0.740, just above the eight-task bound of 0.724, which has already fallen close to ln 2 = 0.693. RTA finds every task meets its deadline, with a largest response of 20 against a deadline of 50, and EDF again passes (simulated over the first 500 of 600 ticks).

In practice the output is meant to be read in a fixed order: total U against 1 first, then the bound, then the RTA table, paying most attention to the tasks with the least slack, which are τ_3 in set B (one tick) and τ_β in set C (two ticks). The simulation panels are a sanity check and not a proof, because they cover only the capped horizon; this is why the verdict for set F rests on RTA and not on the 500-tick run.

Across the six sets the bound was conclusive once (set A), RTA rescued three sets (B, C and F), and DM with RTA handled set E. The bound is therefore a conservative screening test and not a feasibility test, while EDF's $U \leq 1$ test was exact for every implicit-deadline set. In all six cases the simulations agreed with the analytical verdicts.

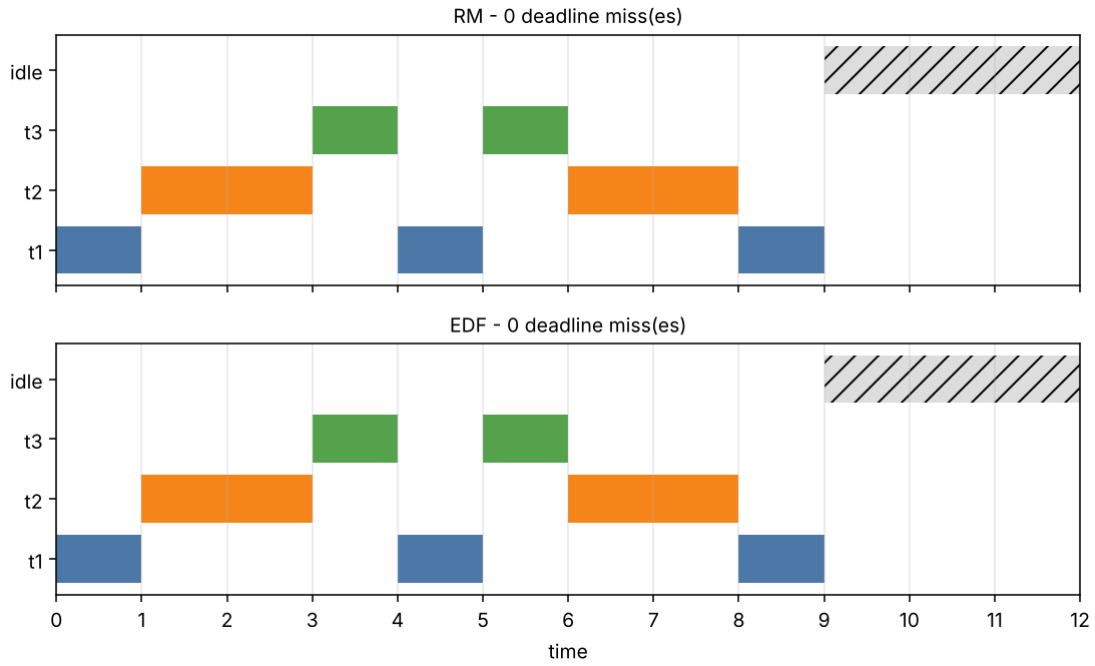


Figure 8: *Set A: RM and EDF schedules (U = 0.750).*

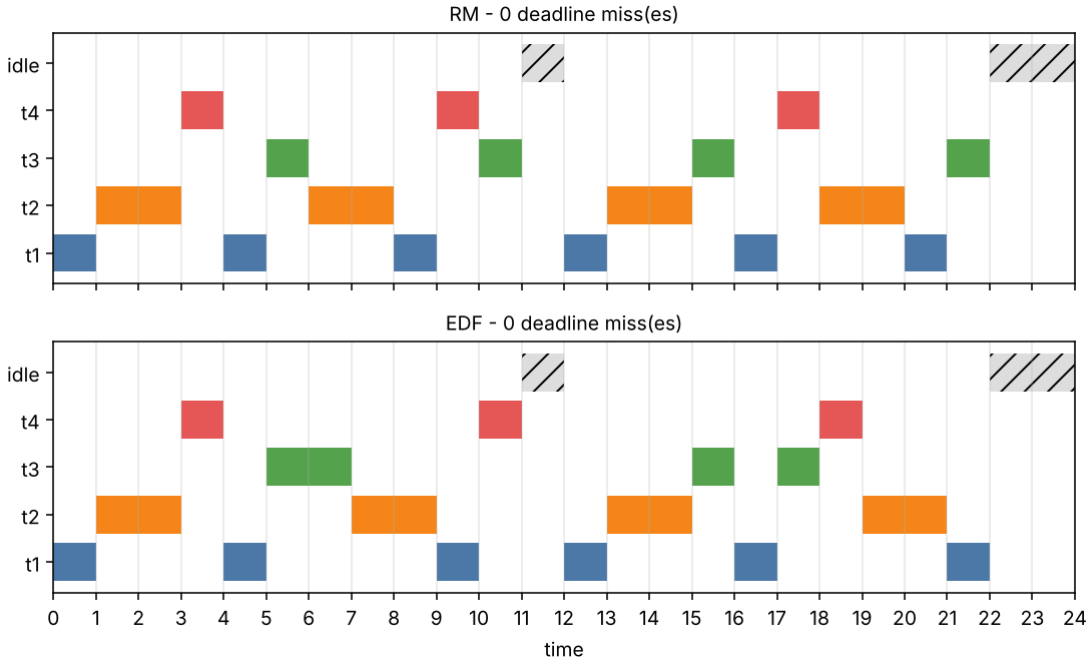


Figure 9: *Set B: RM and EDF schedules (U = 0.875).*

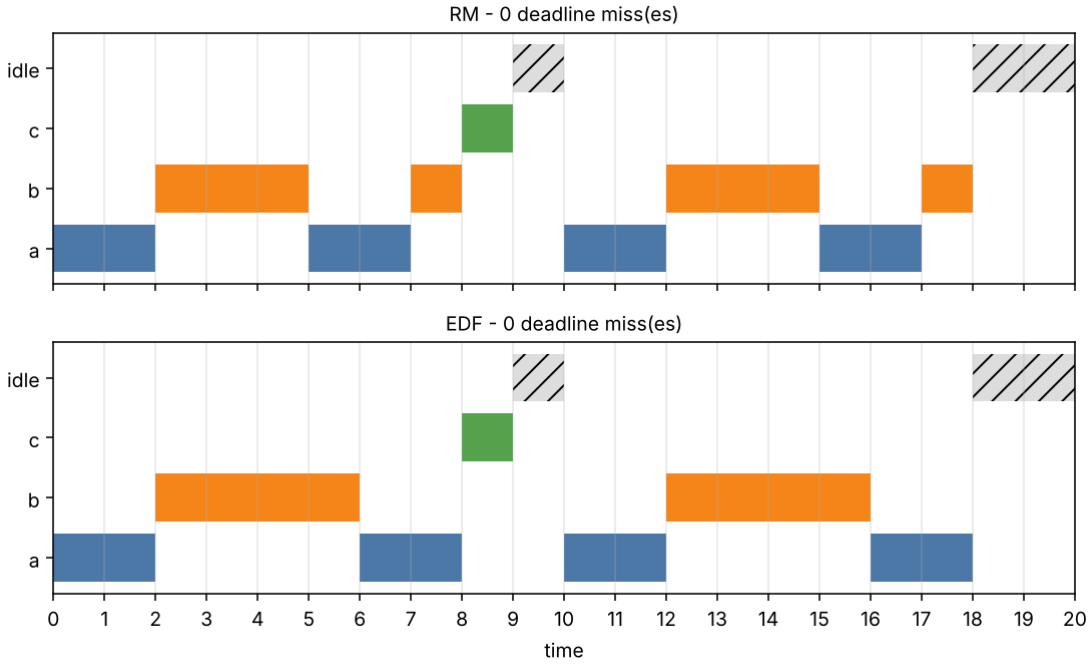


Figure 10: *Set C: the bound fails but RM meets every deadline (U = 0.850).*

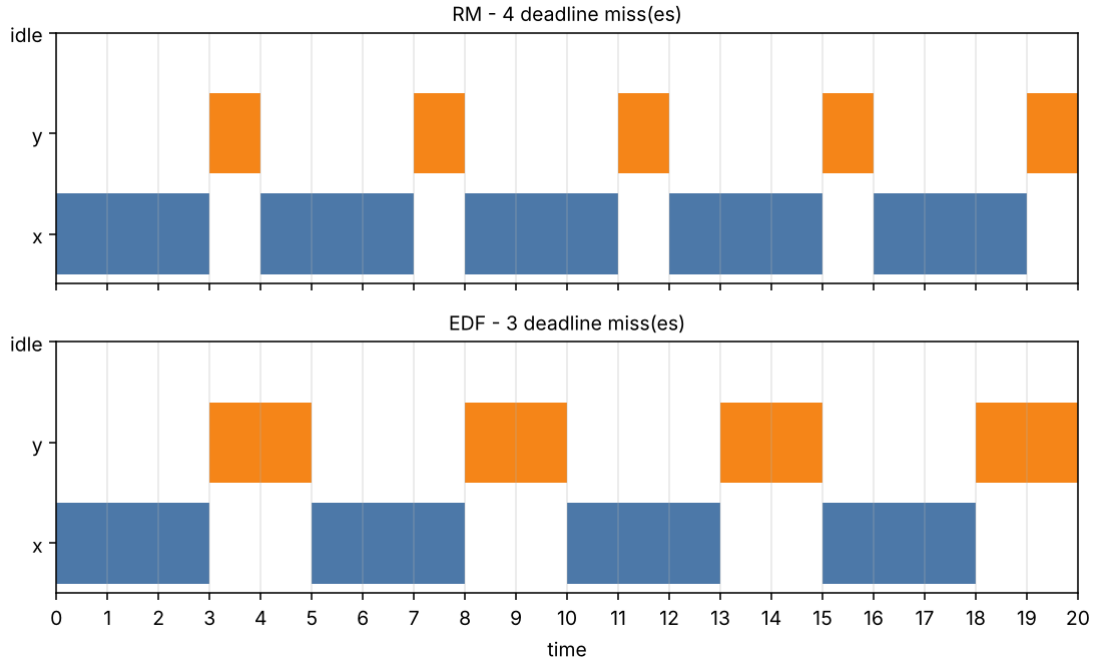


Figure 11: *Set D: overload (U = 1.150); both policies miss deadlines.*

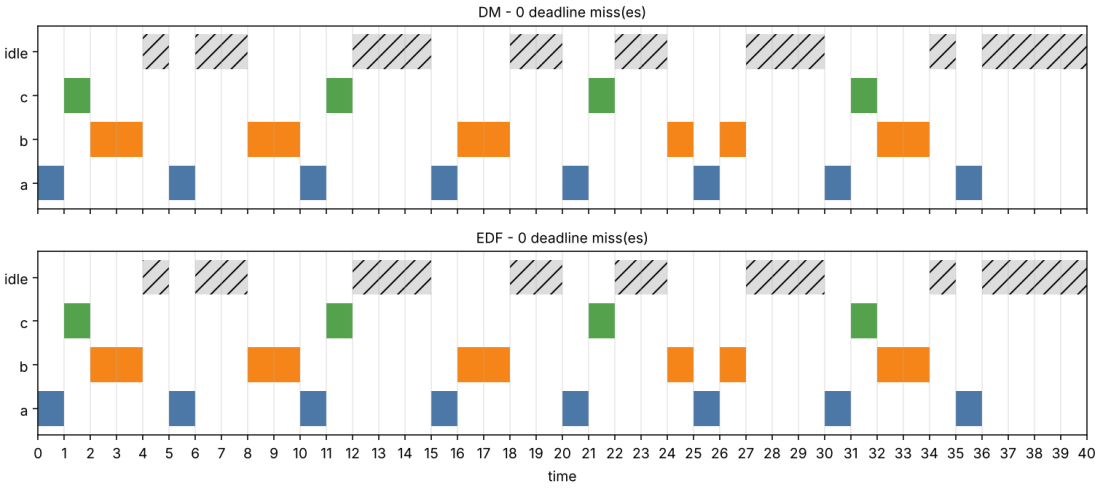


Figure 12: *Set E: constrained deadlines scheduled with DM and EDF.*

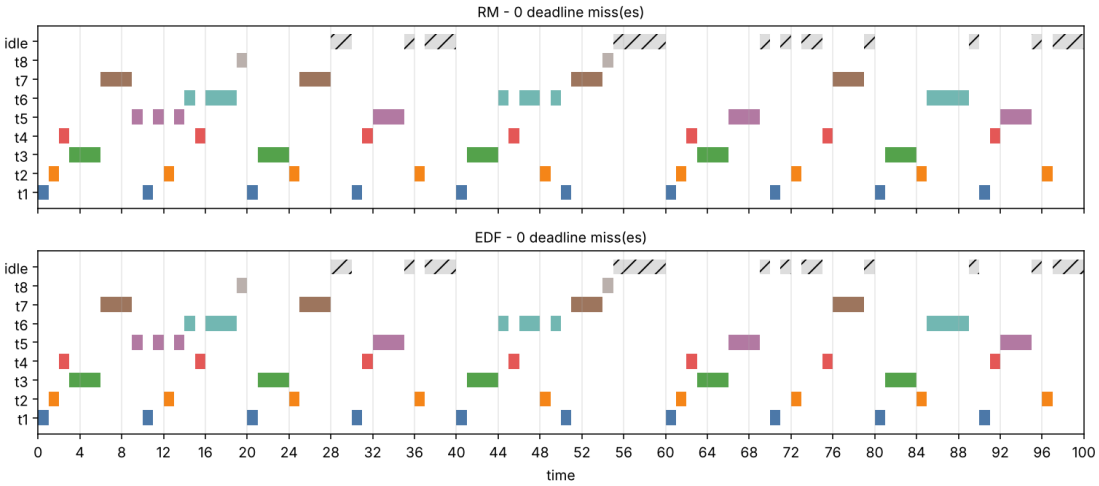


Figure 13: *Set F: eight tasks, first 100 ticks shown (the verdicts in Table 5 use 500 simulated ticks).*

3.5 Reflection: when the bound fails but RTA succeeds

Set C, τ_a (2, 5), τ_β (4, 10) and τ_c (1, 20), has U = 0.850, well above the three-task bound of 0.780, so the

Liu & Layland test is inconclusive and a designer relying on it alone would reject a set that is in fact schedulable. RTA settles the question: R_a = 2, R_β converges in three iterations to 8 (4 → 6 → 8 → 8)

and R_c to 9 (1 → 7 → 9 → 9), all comfortably inside the deadlines of 5, 10 and 20. The bound fails

because it is a worst case over every possible choice of periods, and this set is not a worst case: its periods 5, 10 and 20 are harmonic, each dividing the next, and harmonic sets are known to be RM-schedulable at much higher utilization. Exact tests matter because they remove that pessimism at tiny cost, a few fixed-point iterations per task, and they return actual response times that show the margin left. Without them one would either reject a feasible design or move to EDF unnecessarily and accept its dynamic-priority overhead and poor overload behaviour (set D).

4. Mini Project 2.3: Partitioned Multiprocessor Scheduling Simulator

4.1 Objective

To partition a periodic task set across m cores with four bin-packing heuristics, using the Liu & Layland bound as the per-core admission test, and to compare cores used and the utilization actually achieved.

4.2 Method

- **Admission test:** a task joins a core if the core's load plus u_i is at most `rm_bound(k + 1)`, where k is the number of tasks already on that core. The threshold is recomputed for every core and never replaced by a flat 0.69 or 1.0.
- **Heuristics** (shared signature `heuristic(tasks, m)` returning the cores and the list of unplaced tasks): FF takes tasks in input order and uses the first admissible core; FFD sorts by decreasing u and then applies FF; BFD sorts by decreasing u and chooses the admissible core with the least remaining capacity after placement; WF takes tasks in input order and chooses the admissible core with the most remaining capacity.
- **Reported per heuristic:** whether all tasks were placed, per-core utilization, and achieved total utilization as a percentage of m (counting placed tasks only). The minimum number of cores each heuristic needs is found by increasing m until it succeeds, and is compared with the lower bound ⌈U⌉.
- **Check:** for a successful partition each core is simulated with the per-core RM simulator over the hyperperiod.

4.3 Results

Table 6: Test sets (m = 4 except set 6, which uses m = 3)

Set	Description	Total U	U / m
1 balanced	twelve tasks with u = 0.25	3.000	75%
2 skewed	three large tasks (0.60, 0.60, 0.55) and nine small ones (0.10)	2.650	66%

Set	Description	Total U	U / m
3 tight	four tasks with u = 0.90 and four with u = 0.05	3.800	95%
4 adversarial	five tasks with u = 0.51 (C = 51, T = 100)	2.550	64%
5 order-sensitive	four tasks with u = 0.15 listed before four tasks with u = 0.50	2.600	65%
6 WF-hostile	u = 0.10, 0.35, 0.45, 0.40, 0.50 (T = 100), m = 3	1.800	60%

Table 7: Outcome per heuristic: result and placed utilization as a share of m

Set	FF	FFD	BFD	WF
1 balanced	OK, 75.0%	OK, 75.0%	OK, 75.0%	OK, 75.0%
2 skewed	OK, 66.2%	OK, 66.2%	OK, 66.2%	OK, 66.2%
3 tight	FAIL (4 left), 90.0%	FAIL (4 left), 90.0%	FAIL (4 left), 90.0%	FAIL (4 left), 90.0%
4 adversarial	FAIL (1 left), 51.0%	FAIL (1 left), 51.0%	FAIL (1 left), 51.0%	FAIL (1 left), 51.0%
5 order-sensitive	FAIL (1 left), 52.5%	OK, 65.0%	OK, 65.0%	OK, 65.0%
6 WF-hostile	FAIL (1 left), 43.3%	OK, 60.0%	OK, 60.0%	FAIL (1 left), 43.3%

On set 2 the per-core loads are 0.70, 0.70, 0.75 and 0.50 for FF, FFD and BFD, and 0.70, 0.70, 0.65 and 0.60 for WF. On set 5 FFD, BFD and WF end with 0.65 on every core.

Table 8: Minimum number of cores each heuristic needs, against the lower bound ⌈U⌉

Set	⌈U⌉	FF	FFD	BFD	WF
1 balanced	3	4	4	4	4
2 skewed	3	4	4	4	4
3 tight	4	5	5	5	5
4 adversarial	3	5	5	5	5
5 order-sensitive	3	5	4	4	4
6 WF-hostile	2	4	3	3	4

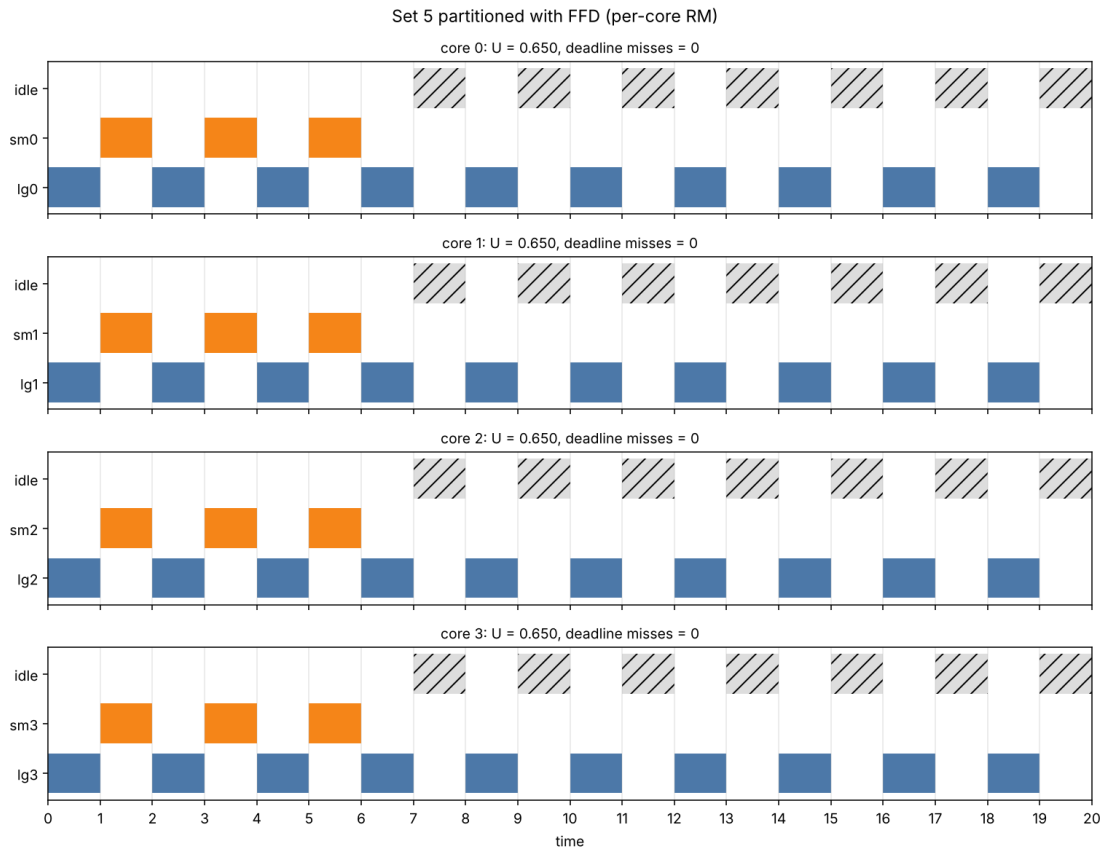


Figure 14: *Set 5 partitioned with FFD: one row group per core, every core meets all deadlines (0 misses, U = 0.650 each).*

4.4 Analysis

Section 2.8.1 reduces partitioned scheduling to m independent uniprocessor problems plus an assignment problem, and the assignment is bin packing, which is NP-hard. Section 2.8.3 therefore uses heuristics, with the per-core RM test as the fit predicate. In our simulator that predicate is not a fixed bin size: a core admits a task if its load plus u_i is at most rm_bound(k + 1), which is 1.000 for the first task but 0.828, 0.780 and 0.757 at two, three and four tasks, so each core's usable capacity shrinks as it fills.

FFD beat plain FF where order mattered. In set 5 four small tasks (u = 0.15) arrive before four large ones (u = 0.5). FF stacks all four small tasks on core 0 (load 0.60), so the large tasks land one per core on cores 1 to 3 and the fourth finds no home, since core 0 would reach 1.10 and the others 1.00, both above 0.828. FFD places the large tasks first, one per core because two cannot share (1.0 > 0.828), and then drops one small task into each, giving 0.65 on all four cores and a complete partition. Placing the biggest items first leaves the small ones to fill the remaining gaps, which is the usual argument for decreasing-order heuristics.

WF always picks the emptiest core, which levels the load but fragments free space, so no core keeps a large hole. Set 6 shows this: with utilizations 0.10, 0.35, 0.45, 0.40 and 0.50 on three cores (total 1.80, only 60% of capacity), WF ends at 0.50, 0.35 and 0.45, and the last task (0.50) fits nowhere because 0.35 + 0.50 = 0.85 exceeds 0.828. FF fails the same set. FFD and BFD succeed with loads 0.60, 0.80 and 0.40. WF is not uniformly worse, since on set 5 its spreading works well, but its result depends on arrival order in a way FFD's does not. BFD matched FFD on all six sets.

The adversarial set (set 4) is five tasks with C = 51 and T = 100 (u = 0.51 each) on four cores. Total utilization is 2.55, only 64% of the ceiling of 4, yet all four heuristics fail, placing four tasks (51% of m) and rejecting the fifth. By hand, two such tasks on one core need 1.02, above both rm_bound(2) = 0.828 and the absolute limit of 1, so each core holds only one and five tasks need five cores. This is the limitation of Section 2.8.1: total utilization below m is necessary but not sufficient, because tasks cannot be split across cores, so any task with u above 0.5 occupies a core largely alone.

Two caveats keep these numbers honest. The Liu & Layland predicate is only sufficient: in set 3 the four tasks with u = 0.90 fill all four cores and the four small tasks are rejected because 0.95 exceeds 0.828, yet an exact RTA test would admit them (a task with C = 1, T = 20 has response time 10 behind one with C = 9, T = 10). Part of that failure comes from the test, not from the packing. Likewise the gap to the ⌈U⌉ lower bound, for example 3 cores against 2 for FFD on set 6, mostly reflects the per-core limit of about 0.76 to 0.83 rather than poor packing.

5. Conclusions

- The simulators reproduce the textbook examples exactly: U = 0.750 and 0.875, bounds 0.780 and 0.757, and RTA response times 1, 3, 4 and 11.
- The Liu & Layland bound is sufficient but pessimistic. RTA settled three of the six task sets that the bound could not, and EDF's U ≤ 1 test was exact for every implicit-deadline set.
- Priority inversion is unbounded without a protocol (9, 13 and 17 ticks of blocking as medium tasks are added) and bounded under PIP (5 ticks every time, below the 6-tick critical section).
- Partitioned scheduling is limited by bin packing: a set at 60% of capacity can fail to partition, FFD and BFD handled the order-sensitive sets that defeated FF, and WF's load balancing fragments free space.

Limitations

- Time is discrete with one-tick resolution and no context-switch cost; a context switch is counted whenever the running job changes between consecutive busy ticks.
- The inversion scenario assumes H needs 2 ticks of work inside the resource; changing that changes the completion time of H but not the blocking time.
- The partitioning test sets were constructed to expose differences between heuristics, not drawn from a real workload.

Delivered files

`rts_sim.py`, `run_project_2_1.py`, `pip_extension.py`, `schedulability_tool.py`, `run_project_2_2.py`, `partition.py`, the printed outputs (`project_2_1_output.txt`, `project_2_2_output.txt`, `project_2_3_output.txt`) and all charts embedded above.